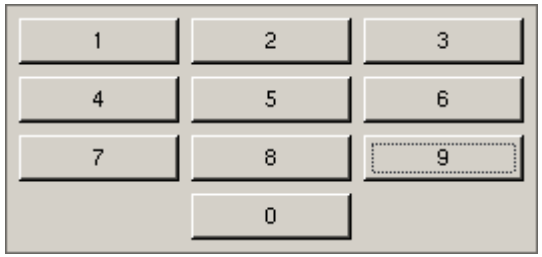
Mapping Many Signals to One

by Jasmin Blanchette

Qt allows us to connect multiple signals to the same signal or slot. This can be useful when we provide the user with many ways of performing the same operation. Sometimes, however, we would like the slot to behave slightly differently depending on which widget invoked it. In this article we explore various solutions, including the use of QSignalMapper.

- The Trivial Solution
- The sender() Approach
- The Subclass Approach
- The Signal Mapper Approach
- The Layout



To illustrate the problem, we will implement a `Keypad` widget that provides ten QPushButtons, numbered 0 to 9, and a `digitClicked(int)` signal that is emitted when the user clicks a button. We will review four solutions and discuss their respective merits.

The Trivial Solution

The most straightforward solution to our problem (but also the silliest) is to connect the ten QPushButton objects' `clicked()` signals to ten distinct slots called `button0Clicked()` to `button9Clicked()`, each of which emits the `digitClicked(int)` signal with a different parameter value (0 to 9). Here's the definition of the `Keypad` class:

```
class Keypad : public QWidget
{
    Q_OBJECT
public:
    Keypad(QWidget *parent = 0);

signals:
```

```
    void digitClicked(int digit);

private slots:
    void button0Clicked();
    void button1Clicked();
    ...
    void button9Clicked();

private:
    void createLayout();

    QPushButton *buttons[10];
};
```

This is the `Keypad` constructor:

```
Keypad::Keypad(QWidget *parent) : QWidget(parent)
{
    for (int i = 0; i < 10; ++i) {
        QString text = QString::number(i);
        buttons[i] = new QPushButton(text, this);
    }

    connect(buttons[0], SIGNAL(clicked()), this, SLOT(button0Clicked()));
    ...
    connect(buttons[9], SIGNAL(clicked()), this, SLOT(button9Clicked()));

    createLayout();
}
```

In the constructor, we create the QPushButtons, and we tediously connect each button's `clicked()` signal to the corresponding private slot.

```
void Keypad::button0Clicked()
{
    emit digitClicked(0);
}

...

void Keypad::button9Clicked()
{
    emit digitClicked(9);
}
```

Each slot simply emits the `digitClicked(int)` signal with a different hard-coded argument.

Needless to say, this approach is inflexible and error-prone. It is practicable for a small number of connections, but even for a 10-key `Keypad`, the copy/paste is almost unbearable. Let's move on to a better solution.

The sender() Approach

The next step is to merge the `button`*N*`Clicked()` slots into one private slot that emits the `digitClicked(int)` signal with the correct parameter, depending on which button was clicked. This is possible using the QObject::sender() function, as we will see shortly. The `Keypad` constructor becomes:

```
Keypad::Keypad(QWidget *parent) : QWidget(parent)
{
    for (int i = 0; i < 10; ++i) {
        QString text = QString::number(i);
        buttons[i] = new QPushButton(text, this);
        connect(buttons[i], SIGNAL(clicked()), this, SLOT(buttonClicked()));
    }
    createLayout();
}
```

And this is the code for the `buttonClicked()` slot:

```
void Keypad::buttonClicked()
{
    QPushButton *button = (QPushButton *)sender();
    emit digitClicked(button->text()[0].digitValue());
}
```

We start by calling `sender()` to retrieve a pointer to the QObject that emitted the signal that invoked this slot. In this particular example, we know that the sender is a QPushButton, so we can cast `sender()`'s return value to a QPushButton `*`. Then we emit the `digitClicked(int)` signal with the digit value shown on the button.

The drawback of this approach is that we need a private slot to do the demultiplexing. The code in `buttonClicked()` isn't very elegant; if you suddenly replace the QPushButtons with another type of widget and forget to change the cast, you will get a crash. Similarly, if you change the text on the buttons (for example, "NIL" instead of "0"), the `digitClicked(int)` signal will be emitted with an incorrect value.

Finally, the use of `sender()` leads to tightly coupled components, which many people consider to be bad programming style. It isn't quite so bad in this example, because `Keypad` already knows about the button objects, but if `buttonClicked()` was a slot in another class, the use of `sender()` would have the unfortunate effect of tying that class to an implementation detail of the `Keypad` class.

The Subclass Approach

Our third approach requires no private slot in `Keypad`; instead, we make sure that the buttons themselves emit a `clicked(int)` signal that can be directly connected to `Keypad`'s `digitClicked(int)` signal. (When connecting a signal to another signal, the target signal is emitted whenever the first signal is emitted.) This requires subclassing QPushButton as follows:

```
class KeypadButton : public QPushButton
{
    Q_OBJECT
public:
    KeypadButton(int digit, QWidget *parent);

signals:
    void clicked(int digit);

private slots:
    void reemitClicked();

private:
    int myDigit;
};

KeypadButton::KeypadButton(int digit, QWidget *parent)
    : QPushButton(parent)
{
    myDigit = digit;
    setText(QString::number(myDigit));
    connect(this, SIGNAL(clicked()), this, SLOT(reemitClicked()));
}

void KeypadButton::reemitClicked()
{
    emit clicked(myDigit);
}
```

Whenever QPushButton emits the `clicked()` signal, we intercept it in our `KeypadButton` subclass and emit the `clicked(int)` signal with the correct digit as the argument.

The `Keypad` constructor then looks like this:

```
Keypad::Keypad(QWidget *parent) : QWidget(parent)
{
    for (int i = 0; i < 10; ++i) {
        buttons[i] = new KeypadButton(i, this);
        connect(buttons[i], SIGNAL(clicked(int)), this, SIGNAL(digitClicked(int)));
    }
    createLayout();
```

```
}
```

This approach is both flexible and clean, but it is quite cumbersome to write, because it forces us to subclass QPushButton.

The Signal Mapper Approach

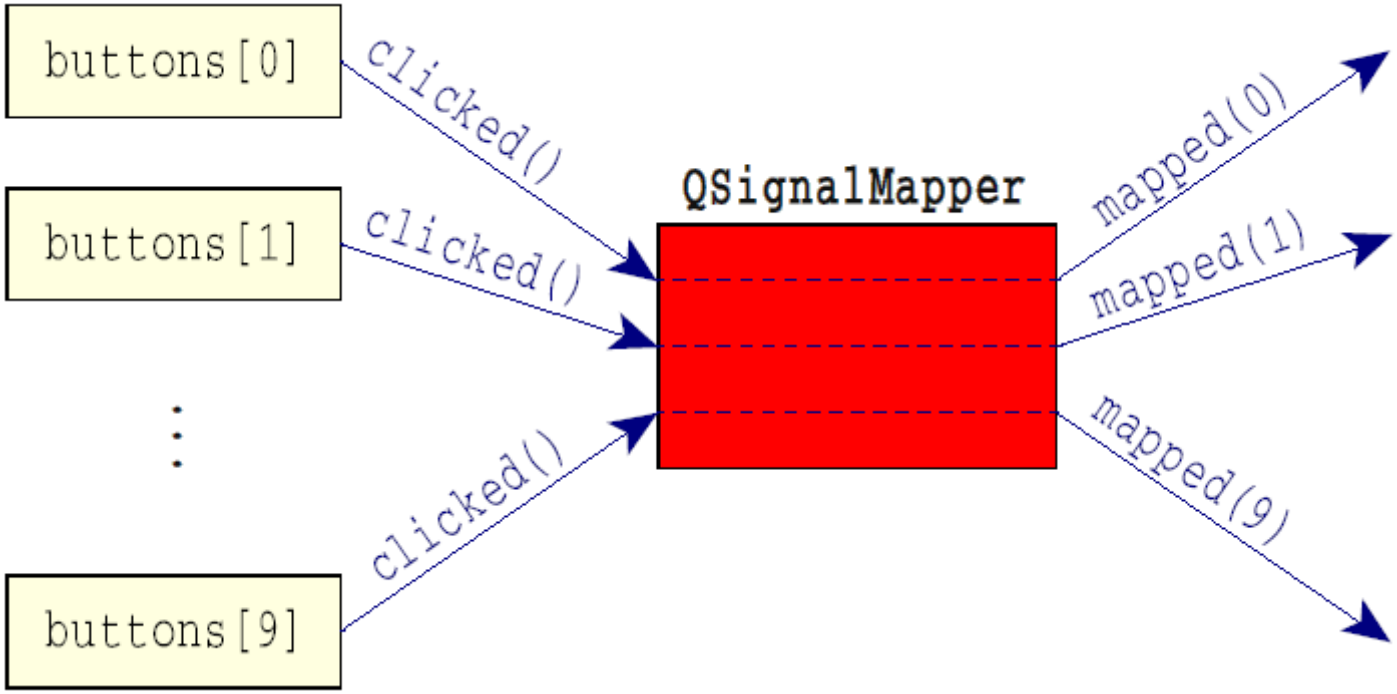
The fourth and last approach does not require any private slots, nor does it need a QPushButton subclass. Instead, the entire signal-related logic is implemented in the `Keypad` class's constructor:

```
Keypad::Keypad(QWidget *parent)
    : QWidget(parent)
{
    QSignalMapper *signalMapper = new QSignalMapper(this);
    connect(signalMapper, SIGNAL(mapped(int)), this, SIGNAL(digitClicked(int)));

    for (int i = 0; i < 10; ++i) {
        QString text = QString::number(i);
        buttons[i] = new QPushButton(text, this);
        signalMapper->setMapping(buttons[i], i);
        connect(buttons[i], SIGNAL(clicked()), signalMapper, SLOT(map()));
    }

    createLayout();
}
```

First, we create a QSignalMapper object. QSignalMapper inherits from QObject and provides a means of establishing a relationship between a set of zero-parameter signals and a one-parameter signal or slot. The call to `setMapping()` inside the `for` loop establishes a mapping between a button and an integer value; for example, `buttons[3]` is associated with the integer value 3.



When the `clicked()` signal of a button is emitted, QSignalMapper's `map()` slot is invoked (thanks to the `connect()` call in the `for` loop). If a mapping exists for the object that emitted the signal, the `mapped(int)` signal is emitted with the integer value set using `setMapping()`. That signal is in turn connected to the `Keypad`'s `digitClicked(int)` signal. The `setMapping()` function exists in two versions: one that takes an `int` and one that takes a QString as the second argument. This makes it possible to associate an arbitrary string with a sender object, instead of an integer. When such a mapping exists, QSignalMapper emits a mapped(const QString &) signal.



QSignalMapper does not directly support any other data types. This means that, for example, if you were to implement a palette tool allowing the user to choose a color from a set of standard colors and needed to emit a colorSelected(const QColor &) signal, your best bet would be to use the `sender()` approach or the subclass approach described above. If you already use a subclass of, say, QToolButton to represent a color, it doesn't cost you much to add a clicked(const QColor &) slot to it.

The Layout

Although this example isn't about layouts, the code won't compile if we don't implement the `createLayout()` function called from the constructor. Here's the missing code:

```
void Keypad::createLayout()
{
    QGridLayout *layout = new QGridLayout(this, 3, 4);
    layout->setMargin(6);
    layout->setSpacing(6);

    for (int i = 0; i < 9; ++i)
        layout->addWidget(buttons[i + 1], i / 3, i % 3);
    layout->addWidget(buttons[0], 3, 1);
}
```

The code for `main()` is also missing; we leave its implementation as an exercise for the reader.



This document is licensed under the Creative Commons Attribution-Share Alike 2.5 license.

Copyright © 2004 Trolltech Trademarks Scripting Qt »